DEDAN KIMATHI UNIVERSITY OF TECHNOLOGY

FINAL REGRESSION TESTING & BUG VERIFICATION REPORT

University Student Course Registration System (Flask & SQLite Database)

Course Code:	CCS 4201 - Software Testing & QA
Lead QA Engineer:	Patrick Muli (Reg: C026-01-0001/2023)
Assessment Date:	August 11, 2026
System Status:	**READY FOR PRESENTATION / ACCEPTABLE FOR DEMONSTRATION**

1. Executive Summary

This report delivers the final results of the corrective action, regression testing, and security verification executed on the DeKUT Student Course Registration System. After the initial QA audit identified 6 critical/high security and integrity defects, a complete FIX → TEST → VERIFY cycle was conducted. All 6 target defects have been successfully resolved, and a complete regression test suite of 30 test cases was re-executed. The overall testing result shows a **100% Pass Rate** across all functional scenarios.

Metric	Before Fixing	After Fixing	Status
Total Test Cases	30	30	Verified
Passed Tests	26	30	100.00% Pass Rate
Failed Tests	4	0	0.00% Failure Rate
Confirmed Defects	6	0	All Resolved
Vulnerabilities	2 Critical / 3 High	0 Remaining	Secured

2. Corrective Actions Implemented

The following engineering modifications were implemented on the Python Flask + SQLite backend service to resolve the defects:

• **BUG-01 (Password Security):** Integrated Werkzeug's secure password hashing utility functions. Passwords are now converted to secure SHA256 hashes during signup and validated via hash checks during login.

• **BUG-02 (API Token Authentication):** Implemented a token session table in SQLite. When authentication succeeds, the client stores a secure bearer token, which is validated against the active session before course viewing, registering, or dropping is permitted.

• **BUG-03 (SQLite Foreign Keys):** Modified database connector to execute PRAGMA foreign_keys = ON; explicitly, enabling referential integrity cascading.

• **BUG-04 (Signup Validation):** Integrated server-side validation rejecting empty, format-incompatible (non-student emails), or weak/short passwords with HTTP 400 Bad Request responses.

• **BUG-05 (Relational Prerequisites):** Developed a relational database checker verifying dynamic prereq requirements on active database enrolments, blocking unauthorized registrations.

• **BUG-06 (NOT NULL Constraints):** Re-structured database table schemas to set critical columns as non-nullable, preventing database corruption.

3. Detailed Bug Fix Verification

BUG-01: Plaintext Password Storage in users table	
Fix Implemented:	Hashed password using Werkzeug's generate_password_hash(). Verification done using check_password_hash() in login API.
Retest Performed:	1. Signup new user. 2. Execute SQLite query 'SELECT password FROM users'.
Evidence & Observed State:	Password field contains hashed scrypt string (scrypt:32768:8:1...)
Result Status:	**FIXED**

BUG-02: Lack of Authentication Token / Session Validation on APIs	
Fix Implemented:	Added a sessions table to store hex tokens. Protected APIs require Bearer tokens in Authorization header.
Retest Performed:	1. Query GET /api/courses without header. 2. Attempt manipulation of user_id query parameters.
Evidence & Observed State:	Without token returns 401. Mismatched token user_id returns 403.
Result Status:	**FIXED**

BUG-03: Foreign Key Constraint Enforcement Disabled	
Fix Implemented:	Executed 'PRAGMA foreign_keys = ON;' on database connection initiation.
Retest Performed:	1. Insert user and enrolment. 2. Delete user and check if enrolment is auto-deleted.
Evidence & Observed State:	SQLite cascading deletions confirmed active. Enrolment count goes to 0.
Result Status:	**FIXED**

BUG-04: Missing Server-Side Signup Validation	
Fix Implemented:	Implemented strict backend validation on signup request payload with length and regex checks.
Retest Performed:	1. POST empty payload to /api/signup. 2. Check for KeyError / 500 error status.
Evidence & Observed State:	API returns 400 Bad Request with a clear message: 'Field is required'. No 500 thrown.
Result Status:	**FIXED**

BUG-05: Simulated Prerequisite Checking Bypass	
Fix Implemented:	Implemented dynamic prerequisite checker that queries student's actual database enrolments.
Retest Performed:	1. Attempt to register for CIT 3108 (requires CIT 3102) without enrolling in CIT 3102.
Evidence & Observed State:	Backend blocks enrolment with 400 error message: 'Prerequisite requirement unmet'.
Result Status:	**FIXED**

BUG-06: Missing NOT NULL Constraints on Enrolment Mapping	
Fix Implemented:	Created database schema migration block to drop and recreate enrolments with user_id and course_id NOT NULL.
Retest Performed:	1. Try to run 'INSERT INTO enrolments (user_id) VALUES (NULL)'.
Evidence & Observed State:	SQLite throws IntegrityError: 'NOT NULL constraint failed: enrolments.user_id'
Result Status:	**FIXED**

4. Regression Testing Results

All 30 test cases were executed to verify system functionality after applying security and data modifications. The following is a subset of key security and database verification test cases:

ID	Scenario Description	Expected Result	Status
TC-01	Successful student signup with valid inputs	Account created, redirected to Login	PASS
TC-05	Successful student login	Success, generates token, redirects to dashboard	PASS
TC-06	Login with invalid password	Login fails, error 401	PASS
TC-17	Register for a course with a schedule clash	Blocked, toast error 'Schedule clash'	PASS
TC-18	Exceed credit limit validation (Max 24 Credits)	Blocked, toast error 'Credit limit exceeded'	PASS
TC-20	Register for a course with unmet prerequisites	■ Prerequisite Required shown, blocked	PASS
TC-27	Access courses list for other users without authentication	API blocks access with 401 Unauthorized	PASS
TC-28	Modify registration status of another user ID	API blocks access with 401 Unauthorized / 403 Forbidden	PASS
TC-29	Sanitize user inputs to prevent XSS payloads	HTML character references encoding or input block	PASS

5. Database & Security Status After Corrective Actions

Security Regression Check:

• Attempting to retrieve courses or modify registrations without a valid Bearer token results in a controlled 401 Unauthorized response.

• Modifying the user_id parameter to access other students' records triggers 403 Forbidden validation checks.

• SQLite direct inspections confirm that all passwords are encrypted via industry-standard scrypt hashing.

Database Integrity Check:

• Cascading deletion checks are 100% active. Deleting a user row automatically cascade deletes matching enrolment link records.

• SQLite rejects all NULL user and course associations inside the enrolments mapping table.

6. Continuous Improvement Recommendations

Identified Issue	Recommended Action / Fix	Priority	Module
HTTPS / SSL Encryption	Expose Flask service through HTTPS using SSL certificates to encrypt tokens in transit.	High	API Security
Database Index Optimization	Add indexes to columns day, slot, and user_id to speed up validation checks at scale.	Medium	Database Operations
Enhanced Password Complexity Rules	Extend server-side validation to check for numbers, upper-case, and special characters.	Medium	Authentication Forms

7. Final QA Assessment & Sign-off

Overall Quality Status: READY FOR PRESENTATION / ACCEPTABLE FOR DEMONSTRATION ■

Summary Assessment:

The system has been successfully fortified and validated. All critical security vulnerabilities (plaintext password storage, lack of API session tokens) and logical defects (prerequisite checking bypasses, foreign key orphans) are fully resolved. Regression testing verifies that all user-facing features (login, signup, courses grid, credit totals, time clash blocks, and dynamic weekly timetable) operate seamlessly without regression. The application is highly stable and acceptable for academic course presentation.